hxp — User Manual

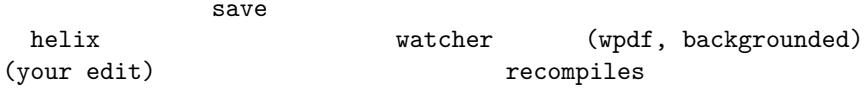
A live-preview workflow for **Markdown**, **LaTeX**, and **Typst**. You edit in helix on the left, a PDF viewer auto-reloads on the right, and every save recompiles. Compile errors are turned into a PDF (and an optional side terminal readout) so you fix and re-save without leaving the editor. Click in the PDF to jump back to the source line.

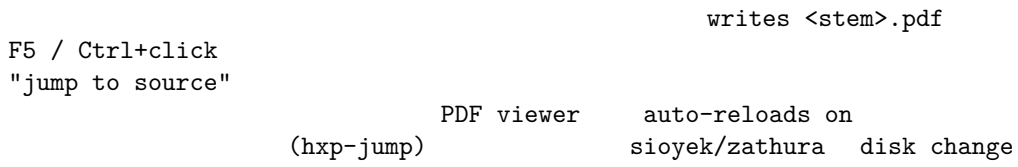
This is the operational reference. For a project overview and the install matrix, see `README.md`.

Contents

Mental model

One `hxp <file>` call starts three cooperating pieces and tears them all down when you quit the editor:





- **Editing** happens in helix; nothing special is required of you — just save.
- **Compiling** is done by a backgrounded watcher (`wpdf`) that fires on each save. Success replaces the PDF; failure renders an *error PDF* in its place.
- **Viewing** is a standard PDF viewer (sioyek or zathura) that reloads the file itself whenever it changes on disk.
- **Inverse search** closes the loop: clicking in the PDF (or pressing **F5**) drives your *already-running* helix to the matching source line.

When you quit helix, `hxp` stops the watcher, closes the viewer window it opened, and sweeps its scratch files — **keeping only the finished `.pdf`**.

Quick start

```
# 1. Confirm the toolchain (run this first whenever something misbehaves)
hxp --doctor

# 2. Open or create a document — tiles editor + viewer, recompiles on save
hxp notes.md
hxp paper.tex
hxp slides.typ

# If the file doesn't exist yet, hxp scaffolds a minimal starter for you.
```

For inverse search and an errors-in-your-eyeline pane, launch inside **tmux** (see the recommended layout).

Install (recap)

```
git clone https://github.com/<you>/hxp.git ~/Applications/hxp
~/Applications/hxp/install.sh
```

Then add one line to `~/.zshrc` and reload your shell:

```
[[ -r "$HOME/.zsh/hxp-main.zsh" ]] && source "$HOME/.zsh/hxp-main.zsh"
```

`install.sh` symlinks the shell functions, the `bin/` helpers, and the viewer configs into place (re-runnable; backs up any real files it would replace). Print the dependency list without installing via `./install.sh --deps`.

PATH matters. The viewers launch `hxp-jump` (and it calls `hxp-mdline`) *by bare name*. Make sure `~/.local/bin` is on `PATH` **before** the viewer spawns, or inverse search silently falls back to opening a fresh editor.

Commands

Three shell functions are added to your shell, plus the `hxp --doctor` diagnostic. All three accept `.md`, `.tex`, and `.typ`.

`hxp <file>`

The full workflow: editor + viewer + watcher, torn down together.

`usage: hxp <file.{md,tex,typ}>  |  hxp --doctor`

What it does, in order:

1. **Scaffolds** the file if it doesn't exist (only for `.md` / `.tex` / `.typ`; any other missing extension is an error, not a scaffold).
2. Resolves the **project root** and therefore the real PDF path. For `.tex` and `.typ` the PDF lands next to the project root, *not* next to an included file you happened to open (see Per-language behavior).
3. Writes a **state file** under `$XDG_RUNTIME_DIR/hxp/` recording the source, PDF, tmux pane, and editor window — this is what inverse search reads.
4. Runs an **initial compile** so the viewer has something to show immediately.
5. **Tiles** the editor terminal to the left half of the work area (floating WMs only — see Window tiling).
6. **Launches the viewer** on the right. For sioyek it opens a dedicated `--new-window` and auto-enables synctex mode so inverse search is armed.
7. **Backgrounds the watcher** (`wpdf -q --no-initial`) and re-focuses helix.
8. **Opens helix.** On a clean first compile it opens just your file. If the first compile *failed*, it also opens the error log (and, for Markdown, the generated `.tex`) as extra buffers so the diagnostics are right there.

When helix exits, `hxp` runs cleanup: stops the watcher, closes *its own* viewer window (via `wmctrl`, so concurrent sessions aren't disturbed), and sweeps scratch files while keeping the PDF.

`hxp --doctor`

A one-screen status of every tool hxp can use and the feature each one enables. **Run it first when something isn't behaving.** `hxp doctor` works too. Real output from a fully-provisioned machine:

```
hxp doctor

Required
    zsh           /usr/bin/zsh
    hx            /usr/bin/hx
    pandoc        /usr/bin/pandoc
    inotifywait   /usr/bin/inotifywait

Compilers
    latexmk       tex + md two-step synctex
    xelatex       unicode/CJK pdf engine
    typst         typst 0.15.0 (3ae52774)

Viewer
    sioyek        active viewer

Watch / reload
    watchexec     debounced watch loop

Inverse search (PDF -> editor)
    hxp-jump      /home/you/.local/bin/hxp-jump
    hxp-mdline    md <- tex line mapping
    tmux          in-place jumps when hxp runs in tmux
    xdotool       X11 keystroke fallback (no tmux)

Window tiling
    session       XDG_SESSION_TYPE=x11
    wmctrl        window placement
    xprop         editor window id
    wm-kind       tiling

Extras
    gawk          awk (mawk/busybox also work)
    cjk-font      Noto Sans CJK KR
```

Marker legend: present/active · optional & absent (feature degraded, hxp still runs) · required & missing (fix this).

`wpdf <file>`

Just the watcher half — recompile on save, no editor and no viewer launched. Useful when you already have an editor and viewer open, or for a headless build loop.

`usage: wpdf [-q] [--no-initial] <file.{md,tex,typ}>`

Flag	Effect
`-q`, `--quiet`	Suppress the per-save `OK`/`ERR` status lines.
`--no-initial`	Skip the first compile (hxp uses this; the initial frame is already rendered).

`wpdf` picks its watch strategy automatically:

- **Typst** uses native `typst watch` (incremental — much faster than full recompiles). Disable with `HXP_NO_NATIVE_TYP=1` to use the generic loop.
- **Everything else** prefers `watchexec` (250 ms debounce, handles editor swap-file churn) and falls back to `inotifywait` when watchexec is absent or `HXP_NO_WATCHEXEC=1` is set.

`wpdf` sweeps its own scratch files on exit (Ctrl-C is safe — it won't take your interactive shell down with it).

`hxp_errs <file>`

A live, full-screen terminal readout of the latest compile state. Run it in a second pane next to `hxp` to keep errors in your eyeline instead of glancing at the PDF. It re-renders on every compile.

`usage: hxp_errs <file.{md,tex,typ}>`

- **OK** state: a green check, the timestamp, and the file path.
- **ERROR** state: the error count (when > 1), the compiler's primary message, the resolved `file:line:col`, the suspect source line, and a ready-to-paste `hx <file>:<line>:<col>` jump target.

`hxp_errs` is a *reader* — it watches the log/markdown/PDF that the watcher writes. It does not compile anything itself, so always pair it with a running `hxp` or `wpdf` for the same file.

The recommended tmux layout

Inverse search lands *in your running helix* only when hxp can reach that helix. The most reliable way is to launch hxp inside **tmux**: `hxp-jump` then drives the exact pane via `tmux send-keys`. Outside tmux it falls back to `xdotool` keystrokes (X11 only), and failing that, spawns a fresh `hx`.

A comfortable two-pane setup — editor+viewer on the left, error readout on the right:

```
tmux new-session \; \
  send-keys 'hxp paper.tex' C-m \; \
```

```
split-window -h \; \
send-keys 'hxp_errs paper.tex' C-m \; \
select-pane -L
```

The viewer window opens as a separate GUI window (tiled right); the `hxp_errs` pane is text inside tmux. You end up looking at: **editor | error readout** in the terminal, with the **PDF** beside them.

Inverse search (PDF → editor)

Jump from a spot in the PDF to the source line that produced it.

Viewer	Trigger
sioyek	Hover the text and press **F5** (one-shot; synctex mode is auto-enabled, no F4 needed). `Ctrl`+click also works.
zathura	`Ctrl`+click on the text.

Under the hood the viewer runs `hxp-jump <file>:<line>[:<col>]`, which:

1. **Maps Markdown back.** If synctex points at the generated ··· `/.hxp_build_<stem>/<stem>.hxp.tex`, it resolves that to your original `<stem>.md` and approximates the line with the shared `hxp-mdline` heuristic.
2. **Finds the session** via the per-source state file hxp wrote.
3. **Delivers the jump** by preference: `tmux send-keys` → `xdotool` keystrokes to the editor's X11 window → spawning a fresh `hx` as last resort.
4. **Handles multi-file projects:** if the synctex-reported file has no exact state file (e.g. an `\input`'d chapter), it matches any live session whose source lives in the same directory tree.

Requirements for in-place jumps: working synctex data, plus either `tmux` (launch hxp inside it) or `xdotool` on X11. Markdown inverse search additionally needs `latexmk` — without it the Markdown path can't emit synctex at all (see below). `hxp --doctor`'s "Inverse search" section tells you which of these are live.

Viewer keybindings

sioyek (custom additions)

These are layered on top of sioyek's stock bindings (which still work — e.g. F8 dark mode, F9 fit-width); they add zathura-style letter keys on keys sioyek left free.

Key	Action
`F5`	**Inverse search** at the cursor → jump to source in helix
`i`	Toggle inverted / dark colors
`a`	Fit page to window **width**
`Ctrl-d` / `Ctrl-u`	Half-page down / up
`Shift-J` / `Shift-K`	Next / previous page (lowercase `j`/`k` stay visual-mark moves)
`H` / `L`	Top / bottom of current page
`'`	Go to mark
`Ctrl-l`	Reload document (auto-reload is also on)

The PDF **auto-reloads** on disk change (`auto_reload_preference 1`), so a successful recompile appears without any keypress. To keep stray launches (e.g. opening a PDF from a file manager) tidy, sioyek is configured single-window / single-instance; `hxp` overrides this per-session with `--new-window` so concurrent sessions on different workspaces each get their own window.

Optional — dual panel. A `_dual_panelify` command (side-by-side variant of the current PDF, via sioyek-python-extensions) is pre-wired in `prefs_user.config` but its keybinding is commented out. To enable it: create the venv and uncomment the `d` binding (see below).

```
venv="${XDG_DATA_HOME:-$HOME/.local/share}/sioyek-extensions"
python3 -m venv "$venv" && "$venv/bin/python" -m pip install sioyek
# then uncomment `_dual_panelify d` in config/sioyek/keys_user.config
```

Override the venv location with `HXP_SIOYEK_VENV`.

zathura

zathura keeps its **stock keybindings**. hxp only configures behavior, not keys: clipboard selection, synctex on (`Ctrl`+click → `hxp-jump`), incremental search, and open-fit-to-width. The PDF reloads automatically on disk change.

Per-language behavior

The three languages share the loop but differ in toolchain, project-root detection, and what makes inverse search work.

Markdown (`.md`)

Pipeline (preferred, when `latexmk` is present):

```
pandoc  md → .hxp_build_<stem>/<stem>.hxp.tex   (LaTeX, in a build dir)
latexmk <stem>.hxp.tex → PDF  (-synctex=1)      (xelatex  lualatex  pdflatex)
```

The two-step path exists so **synctex survives** — that's what makes Markdown inverse search possible (synctex points at the `.hxp.tex`, and `hxp-mdline` maps the line back to your `.md`). Without `latexmk`, hxp falls back to `pandoc` compiling straight to PDF: still renders, but **no working inverse search**.

- **Reader format:** `markdown+tex_math_single_backslash` (so `\(···\)` / `\[···\]` math works).
- **Links** are colored (`colorlinks`, blue).
- **Citations.** If your YAML frontmatter declares `bibliography:`, hxp runs `--citeproc` and lets pandoc load it (it will *not* also pass `--bibliography`, which would double-load and duplicate entries). Otherwise, if a single sibling `.bib` exists next to the source, it's auto-passed as `--citeproc --bibliography=<that.bib>`.
- **CJK.** For Korean/Japanese/Chinese glyphs, hxp auto-selects a CJK font for the xelatex/lualatex engine (so they render instead of vanishing). It probes, in order: *Noto Sans CJK KR*, *Noto Serif CJK KR*, *NanumGothic*, *NanumMyeongjo*, *Baekmuk Batang*. Override with `HXP_CJK_FONT="Your Font"`. If your frontmatter already sets `CJKmainfont:`, hxp leaves it alone.

LaTeX (`.tex`)

```
latexmk -pdf -synctex=1 <root.tex>   (pdflatex; -bibtex if a sibling .bib exists)
```

- **Engine:** plain `latexmk -pdf`, i.e. **pdflatex**. If you need xelatex/lualatex for a `.tex` document (Unicode, fontspec, …), configure it yourself — a `latexmkrc`, or a `% !TEX program = xelatex` line your latexmk respects. (The xelatex engine hxp auto-selects applies to the *Markdown* path and to error PDFs, not to `.tex`.)
- **Project root.** If the file you open contains `\documentclass`, it *is* the root. Otherwise hxp walks up the directory tree looking for a `main.tex`, `root.tex`, or `thesis.tex` that has a `\documentclass`, and compiles that — so you can open and edit an `\input`'d chapter and still get the whole document's PDF. The PDF and synctex sidecar live next to the root.
- **Bibliography.** A sibling `.bib` enables `-bibtex` automatically.

Typst (`.typ`)

```
typst watch  --root <root_dir> <root.typ> <pdf>     (incremental; the default)
typst compile --root <root_dir> <root.typ> <pdf>    (one-shot / generic loop)
```

- **Native watch.** Typst's own incremental `typst watch` drives the loop by default — fast, and it keeps the PDF current. hxp consumes its status stream so that **compile errors also surface** as an error PDF + `hxp_errs` flip (previously typst errors were silent and the viewer kept the last good PDF). Set `HXP_NO_NATIVE_TYP=1` to use the generic full-recompile loop instead.
- **Project root.** hxp walks up for a `main.typ` (or a `typst.toml` marking a package root), and uses `--root` so cross-file `#import`s resolve. The PDF lands next to the root.
- **No synctex.** Typst doesn't emit synctex, so **inverse search isn't available for `.typ`**. Everything else (live reload, error PDFs, the `hxp_errs` readout) works.
- **No LaTeX needed.** Typst error PDFs are rendered *with typst itself*, so `.typ` users don't need a LaTeX toolchain at all.

Error reporting

A failed compile is uniform across all three languages: hxp renders an **error PDF** in place of the document and flips `hxp_errs` to **ERROR**. The viewer keeps showing a PDF — now the error one — so you never stare at a stale page wondering if the save took.

The error PDF leads with a red **"Compile failed"** banner (with an approximate error count when more than one), then:

- **Look Here First** — the compiler's primary message, the resolved `file:line:col` (as a clickable link), and a copy-paste **helix target**.
- **Suspect Line** — the exact source line the error points at.
- **Nearby Source** — a few lines of context (for `.tex` / `.typ`).
- **Compiler Extract** — a windowed slice of the raw log (capped so a cascading LaTeX failure can't produce a 50-page error PDF).
- **(Markdown only) "Where this really is"** — because pandoc errors reference the *generated* LaTeX, hxp shows the accurate `.tex` line **and** a heuristic best-guess at the corresponding Markdown line.

Notes:

- **Typst** halts at the first error by default (so you usually fix one, save, repeat). **pandoc/LaTeX** can cascade, hence the multi-error count and the windowed extract.
- Error PDFs use a **plain hyphen** in the header and conservative packages so they compile even on a pdflatex-only setup.

Environment variables

Variable	Effect
`HXP_VIEWER`	Force `sioyek` or `zathura` instead of auto-detect (sioyek preferred when both present).
`HXP_CJK_FONT`	Override the CJK font family for Markdown PDFs.
`HXP_NO_NATIVE_TYP=1`	Use the generic compile loop instead of `typst watch` (full recompiles; same error surfacing).
`HXP_NO_WATCHEXEC=1`	Use `inotifywait` instead of `watchexec` for the generic loop.
`HXP_NO_TILE=1`	Disable `wmctrl` tiling of the editor / viewer windows.
`HXP_WM`	Force `tiling` or `floating` instead of auto-detect (see below).
`HXP_SIOYEK_VENV`	Path to the sioyek-python-extensions venv (default `${XDG_DATA_HOME:-~/.local/share}/sioyek-extensions`).
`HXP_AWK`	Force a specific `awk` binary for log parsing (mainly for the tests).

Set inline for one run (`HXP_VIEWER=zathura hxp notes.md`) or export from `~/.zshrc` to make it your default.

Internal (set by hxp, not for you): `HXP_VIEWER_PID`, `HXP_VIEWER_KIND`.

Window tiling

Auto split-screen (editor left, viewer right) needs **X11** plus `wmctrl` and `xprop`. Check your session:

```
echo "$XDG_SESSION_TYPE"    # x11 or wayland
```

- **Wayland / no wmctrl:** the tiling code silently no-ops. `hxp` still works — viewer launches, watcher recompiles, error PDFs render, inverse search works. You just place the windows yourself. `HXP_NO_TILE=1` skips it explicitly.
- **Floating WMs** (xfwm, mutter-on-X11, kwin, openbox, …): `hxp` uses `wmctrl -e` to put helix on the left half and the viewer on the right half of the active monitor's work area.

- **Tiling WMs** (i3, sway, bspwm, dwm, awesome, xmonad, qtile, herbstluftwm, river, hyprland): geometry calls are **skipped** — managed containers ignore `_NET_MOVERESIZE_WINDOW` anyway. The viewer opens while helix is focused and the WM's own logic (e.g. autotiling on i3) splits it as a sibling.

Detection is automatic (`wmctrl -m`, falling back to `_NET_WM_NAME`). Override with `HXP_WM=tiling` or `HXP_WM=floating` if the guess is wrong.

Runtime & output files

Created next to your source during a session. **All are swept on exit except the rendered `.pdf`**, which is kept.

Path	Role
`<stem>.pdf`	**The output — kept.**
`<src-dir>/.<stem>.error.log`	Raw compiler stdout/stderr.
`<src-dir>/.<stem>.error.md`	Markdown that gets rendered into the error PDF.
`<src-dir>/.<stem>.debug.tex`	pandoc's md→LaTeX output (Markdown path / failure fallback).
`<src-dir>/.<stem>.tmp.pdf`	Staging path before the atomic move to the real PDF.
`<src-dir>/.hxp_build_<stem>/`	latexmk build tree (`.tex`; and Markdown's synctex intermediate `<stem>.hxp.tex`).
`<pdf-dir>/<stem>.synctex.gz`	Synctex sidecar (kept only while viewing).
`${XDG_RUNTIME_DIR:-/tmp}/hxp/<sha1>.state`	Per-source state file `hxp-jump` reads for inverse search.

The leading dots keep the scratch files out of `ls` and most file pickers. If a crash ever leaves them behind, they're safe to delete — the next `hxp` run recreates whatever it needs.

Troubleshooting

Start with `hxp --doctor` — most problems are a missing optional tool, and doctor names the exact feature each one gates.

Symptom	Likely cause & fix
PDF doesn't update on save	Watcher or viewer reload stalled. Confirm `inotify-tools` (and ideally `watchexec`) via doctor. On a network/odd filesystem, try `HXP_NO_WATCHEXEC=1`.
Inverse search opens a *new* editor instead of jumping	hxp can't reach your helix. Launch hxp **inside tmux**, or install `xdotool` (X11). Also confirm `~/.local/bin` is on `PATH` *before* the viewer starts, so `hxp-jump` resolves.
Inverse search does nothing in a `.md`	Markdown synctex needs `latexmk`. Without it the Markdown path can't emit synctex. (`.typ` has no synctex at all — that's expected.)
Windows don't tile	You're on Wayland, or missing `wmctrl`/`xprop`, or on a tiling WM (it defers to the WM). See Window tiling; override with `HXP_WM`.
CJK glyphs missing in a Markdown PDF	Install `fonts-noto-cjk` or set `HXP_CJK_FONT="Your Font"`. Confirm the resolved font in doctor's *Extras* row.
`.tex` Unicode/fontspec fails under pdflatex	`.tex` compiles with `latexmk -pdf` (pdflatex). Configure xelatex/lualatex yourself via `latexmkrc` or a `% !TEX program` directive.
Bibliography entries appear twice	Don't declare `bibliography:` in YAML *and* rely on a sibling `.bib` — pick one. With the YAML key present, hxp uses `--citeproc` alone.
Typst errors seem silent	They shouldn't be — the error PDF + `hxp_errs` flip should fire. If you set `HXP_NO_NATIVE_TYP=1`, the generic loop is in play; either way errors surface.
Wrong viewer launches	Set `HXP_VIEWER=sioyek` or `HXP_VIEWER=zathura`.
Second `hxp` hijacks the first's window	Shouldn't happen — sioyek sessions get `--new-window` and are matched by PDF basename. If it does, check you're not forcing single-window via a custom sioyek config.

Scratch `.<stem>.*` files left behind after a crash	Safe to delete; the next run recreates them.

Development

```
zsh test/parse-test.zsh    # compiler-log parsers — needs no compilers (fast gate)
zsh test/smoke.zsh         # real compile per language (+ typst watch); skips absent tools
```

`parse-test.zsh` runs in CI under both gawk and mawk to keep the awk log parsers POSIX-clean. `smoke.zsh` exercises the real pipeline and skips any language whose tools aren't installed, so it's useful with a partial toolchain. Both run on every push via `.github/workflows/ci.yml`.

Source map (see `README.md` for the full tree):

File	Role
`zsh/hxp-main.zsh`	`hxp()` / `wpdf()` entrypoints, window tiling, viewer launch.
`zsh/hxp-lib.zsh`	Compile helpers, error rendering, `hxp_errs`, `--doctor`. Sourced by the watcher too.
`bin/hxp-compile`	Thin per-save wrapper watchexec calls (avoids re-sourcing `.zshrc`).
`bin/hxp-jump`	Synctex inverse-search shim (tmux / xdotool / fresh-hx).
`bin/hxp-mdline`	Shared md←tex line-mapping heuristic (used by jump *and* error renderer).
`bin/hxp-dual-panelify`	PATH-resolved wrapper for sioyek's dual-panel extension.

Uninstall

The installer only creates symlinks; remove them and the source line.

```
rm -f ~/.zsh/hxp-main.zsh ~/.zsh/hxp-lib.zsh \
      ~/.local/bin/hxp-compile ~/.local/bin/hxp-jump \
      ~/.local/bin/hxp-mdline ~/.local/bin/hxp-dual-panelify \
      ~/.config/zathura/zathurarc \
      ~/.config/sioyek/prefs_user.config ~/.config/sioyek/keys_user.config
```

Then delete the `source "$HOME/.zsh/hxp-main.zsh"` line from `~/.zshrc`. Any `*.bak.<timestamp>` files the installer made are your originals — restore or discard them as you like.